Using AI and Quarto to Transform Higher Education Data Workflows

Workshop handout

Cecile Gleizes and Claire Heacox, Auburn University Office of Institutional Research

October 2026

This handout was written in Quarto and rendered to PDF with Typst. The source is `handout/handout.qmd` in the project. Everything in it applies to the document you are holding.

Getting in

Two doors, same room. Pick one. Switch if it does not work.

Door 1: RStudio Desktop on your laptop

1. Open the project: double-click `IR-Quarto-Workshop.Rproj`.
2. Files pane (bottom right): `setup/00_install_packages.R`. Click **Source**. Wait for "Done".
3. `setup/01_check_setup.R`. Click **Source**. Ends with "Ready" or tells you what to fix.
4. `exercises/00_hello.qmd`. Click **Render**. A web page opens. You are in.

Door 2: Posit Cloud, in a browser

1. Free account at **posit.cloud**.
2. Open the space link below. Click the project. Wait while it copies. Everything is installed.
3. `exercises/00_hello.qmd`. Click **Render**. You are in.

Door 2 is for today's public data only. Posit Cloud runs on servers you do not control, hosted on AWS. Posit describes itself as a data processor and will provide a data processing addendum on request, but we found no public FERPA or SOC 2 statement for the Cloud product, and no agreement between Posit and your institution exists unless your institution made one. Student records do not go there without your own institution's review and sign-off. The desktop stays on your machine, behind your institution's controls. That is why it is the door you take home.

Posit Cloud space link: _____

Before anything else

Your UNITID: _____ (search `data/institutions.csv` by name; Auburn is 100858)

An AI assistant you can log into: ChatGPT, Claude, Copilot, Gemini. Any.

If you are already in: turn to "The STATS report" and start exercise 1. Do not wait.

What you will leave with

A report about your own institution built from the Department of Education's 2026 Program Performance Data, the preview of the earnings accountability results that become official in February 2027. The code, the data, the documentation, and the output are one file. When the official file arrives, one script changes and your report re-renders.

Why

The problem. More reports, more evidence, more dashboards, more documentation. Same people. The production model has to change, not the hours.

The idea. Put the query, the code, the explanation, the tables, the charts, and the output format in one plain-text file. Render it. Version it. Hand it over. Re-run it next year.

The principle. *The logic is readable; the data never is.* Everything about how a number is made lives in a file anyone can read, including an AI. Student records stay in the system of record and are read at render time.

The landscape, and why these doors

Where people are	Good at	Weak at
Excel, Access	one-off answers, familiarity	repetition, history, logic hidden in cells
Qlik, Tableau, Power BI	dashboards, drill-down	narrative, PDFs, documentation, version control; licensed per seat
SAS, SPSS, Stata	scripted analysis	the report is still assembled by hand; licensed
Jupyter, Colab, Databricks	Python exploration	publishing; Quarto renders notebooks too
VS Code, Positron	multi-language IDEs	newer; Positron is Posit's next IDE, worth watching
RStudio Desktop	free, stable, Quarto built in, one-button render, the IR community	needs an install
Posit Cloud	nothing to install; a browser	1 GB on the free plan; cannot see your warehouse
Posit Workbench and Connect	the server versions: shared, scheduled, published	enterprise licenses

What Auburn did

For twenty years an Access database produced each year's college transfer feedback reports. In 2026 the logic was read out of the VBA into plain rules, re-implemented in Oracle and a Quarto template, and the old and new systems were run on the same cohort and compared figure by figure. The comparison matched, and found three errors the Access report had carried for years. Then the Quarto template rendered one PDF per college, 46 of them, in one run.

What is inside RStudio

More than R. Point at this page during the tour.

Languages, in the same document

Chunk	Runs	Package
`{r}`	R	
`{python}`	Python	`reticulate`
`{sql connection=con}`	SQL against a live connection; result becomes a data frame	`DBI`, `odbc`
`{ojs}`	Observable JavaScript, interactive in the browser	
`{bash}`	shell	

Quarto output formats, one line each

`html` `typst` (PDF, nothing to install) `pdf` (LaTeX) `docx` `pptx` `revealjs` `dashboard` `gfm` (GitHub) and whole websites, books, blogs, manuscripts. Parameters. Conditional content per format. Cross-references. Citations with Zotero. `brand.yml` for institutional colors, fonts and logo in one file.

Packages worth knowing, by job

Job	Packages
Wrangle	`dplyr`, `tidyr`, `janitor`, `stringr`, `lubridate`
Tables	`gt` (HTML, PDF, Word), `kableExtra` (PDF), `flextable` (Word, PowerPoint), `reactable`, `DT` (interactive)
Charts	`ggplot2`, `patchwork`, `ggrepel`, `plotly` (interactive), `leaflet` (maps)
Files	`readxl`, `writexl`, `openxlsx`, `haven` (SAS, SPSS, Stata), `jsonlite`, `httr2` (APIs)

Job	Packages
Databases	`DBI`, `odbc`, `dbplyr` (write dplyr, get SQL), `keyring` (passwords out of files)
Office files	`officer` (build Word and PowerPoint from R)
Interactive	`shiny`, Shinylive (Shiny inside a static page)
Workflow	Projects, `renv` (lock versions), the Git tab, Terminal, Background Jobs, `styler`, `esquisse`
Higher ed	`educationdata` (IPEDS by API), `tidycensus`, `rscorecard`, `IPEDSuploadables`
AI	GitHub Copilot (built into RStudio; subscription), `ellmer` (call a model from code), `chattr`

Publishing

`quarto publish quarto-pub` (free, public). GitHub Pages. Netlify. Posit Connect (licensed). Drop the HTML on SharePoint. Or what Auburn did: an Azure static site behind campus sign-in, with a dashboard that shows each person only the reports they may open.

How IR offices use this

Fact books as websites. Program review profiles, one per department. Course evaluation reports, one per instructor. Common Data Set and IPEDS preparation. Accreditation narratives with the evidence tables embedded. Survey reports from the Qualtrics API. Retention dashboards. Text analysis of comments. Feedback reports to feeder institutions, one PDF each.

IPEDS, question by question

Cecile's segment. Notes space.

The STATS report: exercises 1 to 3

Exercise 1: your institution

1. Open `exercises/01_my_institution.qmd`.
2. Near the top, change `unitid: 100858` to your UNITID.
3. Click **Render**. Read it. Every number came from `data/`; every table and chart from a code chunk you can see.

Then look at the file. The YAML block at the top is settings. `params` is the one thing that changes between institutions. Each grey block is a chunk; `#| echo: false` hides the code from the reader. Everything else is Markdown.

Now click Source, then Visual. The two buttons at the top left of the editor. Same file, two views. Source is the plain text: the marks, the fences, the YAML. Visual looks like a word processor: headings are big, tables are grids, chunks are boxes. People write in Visual; an AI writes Source, because Source is what it was trained on and what you paste into it. Learn to read both. When an assistant hands you a chunk, you are reading Source; when you fix the sentence above it, you are probably in Visual. Nothing is lost switching, ever.

Three verbs that are not the same thing.

Verb	Means	Where
Run	execute one chunk, or one line, in the Console, right now; the output appears below the chunk; nothing is written to disk	the green arrow on a chunk; Ctrl+Enter on a line
Render	execute every chunk from the top in a fresh session and build the output file (HTML, PDF, Word)	the Render button; `quarto render` in the Terminal
Publish	copy a rendered output somewhere other people can open it	`quarto publish`; or drop the file on a server, SharePoint, the portal

Run is for thinking. Render is for the document. Publish is for the audience. A chunk that Runs but does not Render was depending on something in your session that the document never loaded; Render is the test.

Exercise 2: the same report, four ways

Change the `format:` block and Render each time. Change it back after.

Want	Write
Word	`format: docx`
PDF	`format: typst`
Slides	`format: revealjs`
All three at once	list them, then in the Terminal tab: `quarto render exercises/01_my_institution.qmd`

Notice the last section, "Every current program", is missing from the PDF. In the file it is wrapped in `::: {.content-visible when-format="html"}`. Long tables for the web, short ones for paper, one source.

Exercise 3: five institutions, one click

1. Open `exercises/03_render_peers.R`.
2. Edit the list of names and UNITIDs. Change `format` if you want PDFs.
3. Click **Source**. Five reports appear in `output/`.

This is the whole trick behind one-PDF-per-college, one-report-per- department, one-profile-per-program. The template does not know how many there are.

Exercise 6: the IPEDS pattern (for later, or for the fast)

1. Open `templates/ipeds/EF_fall_enrollment.qmd`. Render it. It runs on 3,000 invented students, so it works anywhere.
2. Read one question section: the IPEDS question quoted, the rule in prose, the chunk, the answer. Four pieces, every question.
3. Find the checks section. Change a rule so a check fails. Render. It refuses. That refusal is the feature.

`docs/ipeds-tutorial-for-cecile.md` explains the pattern: one population query, many small question chunks, checks that stop the render, an upload file written by the last chunk, and the year as a parameter. Replace the fake-data chunk with your census query and the rest stays.

If it breaks

Message	Means
`No programs found for unitid`	typo, or your campus reports under a parent OPEID; search `institutions.csv` by name
`could not find function`	a package is not loaded; add `library(...)` in the setup chunk
`object 'x' not found`	a chunk above did not run; Render runs them in order from the top
the PDF table is cut off	shorten column labels or drop a column; or keep it HTML-only

The data, and exercise 4

What PPD:2026 is

Published by the Department of Education in December 2025, updated January 8, 2026, for the AHEAD negotiated rulemaking. One row per **institution × credential level × 4-digit CIP**: 209,321 programs at 5,096 institutions. A preview, for information only. The official results under STATS (the Student Tuition and Transparency System, from the One Big Beautiful Bill Act) arrive **February 2027** at the 6-digit CIP level.

The tables in `data/`

File	One row per	Use for
`ppd_programs.rds`	program	today: earnings, debt, benchmark, pass/fail, completer demographics
`ppd_yearly.rds`	program × award year	2015-16 to 2024-25: enrollment, completers, Pell, every loan type
`ppd_institution_yearly.rds`	institution × year	in-state and out-of-state Title IV enrollment
`ppd_cip6_earnings.rds`	program at 6-digit CIP	the earnings detail the official file will use
`institutions.csv`	institution	find a UNITID; compare institutions
`codebook.csv`	column	every column, what it means, where it came from

Three things to know before computing anything

1. **NA means suppressed, not zero.** Fewer than 10 students (16 for earnings, 30 for FAFSA-based columns) is blanked. Sums undercount.
2. **Counts of 10 to 19 are shown as 15.** Rates built on them use 15.
3. **The dollars are different years.** Earnings and benchmarks: 2024 dollars. Debt: 2019 dollars. Yearly aid: nominal.

The test, in one line

`md_earn_wne_p4` (median earnings of completers working four years after the 2017-18 and 2018-19 cohorts) against `earn_bnchmrk_cip2_wageb` (high school graduates aged 25 to 34 in the state for undergraduate programs; bachelor's holders in the field for graduate programs). Below it: `earnings_test = "Fail"`.

Exercise 4: daydream

Pick one; add a section to your report. Code for six of them is in `solutions/04_daydream_examples.qmd`.

Who is within 10% of the line, either side. Which benchmark each program was held to. Ten years of Pell share. Loan volume by year, Parent PLUS vs Grad PLUS. Non-completion by program, trended. Debt against earnings, one point per program. The same program at five peers. Pass and fail across your whole state by control. Completer demographics against results. Delinquency and default for the one cohort that has it. Your programs at 6 digits. The one-page Typst brief for the provost.

Asking an AI, safely

The rule: share the shape of the data, never the rows.

Column names, types, a description, the codebook: yes. Student records: never. Small aggregates can identify people: think first. Passwords and keys: never.
Today's dataset is public and has no student-level data, so you may paste rows from it. That is the exception. Your warehouse is not like this.

Prompt 1: describe, then ask

> I am writing a Quarto report in R using dplyr, ggplot2 and gt. I have a data frame `tested`, one row per academic program, with these columns: *[paste lines from codebook.csv]*. Write one R chunk that *[what you want]*. Use `theme_minimal()`. Do not install packages. Return only the chunk.

Prompt 2: the error

> Here is my R chunk: *[chunk]*. Here is the error: *[the red text]*. What is wrong and what is the fix?

Prompt 3: make it say something

> Here is a table of results: *[5 to 15 rows]*. Write three sentences for a provost that state the main finding plainly, with no adjectives, and name the one program that most needs attention.

Treat the answer as a draft. Check every number.

Prompt 4: the report you actually owe someone

> I produce *[report]* for *[audience]* every *[cadence]*. Today I build it in *[tool]* by *[steps]*. The data has these columns: *[names and types, no rows]*. Outline a Quarto document that produces it, with a parameter for *[what changes each time]*, and list the chunks I would need.

What the assistant is good and bad at

Good: syntax you forgot, a first draft of a chunk, explaining an error, turning a table into sentences, suggesting a chart. Bad: knowing your data, knowing your rules, arithmetic you did not check, anything it cannot see. It will produce confident code that runs and is wrong. Rendering is the test; reading is the second test.

With an API key (optional)

The `ellmer` package calls a model from an R chunk, so a summary paragraph can be written at render time. Costs money per call. Key in `keyring`, never in a file. Same rule applies: whatever is in the data frame you pass leaves the building.

Cheat sheet, and taking it home

The top of a Quarto file

```yaml
---
title: "Report"
date: today
format:
  html:
    toc: true
    embed-resources: true     # one file, images inside
params:
  unitid: 100858
execute:
  echo: false                 # hide code from readers
  warning: false
---
```

In a chunk

```
#| label: name          # unique; shows in Render progress
#| fig-height: 4
#| echo: true           # show this one chunk's code
```

Markdown you will use

`# Heading` `## Subheading` `**bold**` `*italic*` `` `code` `` `[text](url)` `1. list` `- bullet` `| a | b |` for tables, `::: {.callout-note}` for a box, `` `r params$unitid` `` for a value inside a sentence.

Render many

```
quarto::quarto_render("report.qmd", execute_params = list(unitid = 100858),
                      output_file = "Auburn.html")
```

Connect to your own warehouse (desktop only, at home)

Not in Posit Cloud. See the note under Door 2.

```
con <- DBI::dbConnect(odbc::odbc(), dsn = "MY_DSN",
                      UID = "my_user", PWD = keyring::key_get("my_service",
"my_user"))
df  <- DBI::dbGetQuery(con, "SELECT ... FROM ...")
```

`MY_DSN` is an ODBC data source your IT office sets up on your machine. Set the password once in the Console: `keyring::key_set("my_service", "my_user")`. It is stored by the operating system, not in any file, and never in a document like this one.

February 2027

The official STATS file will arrive at 6-digit CIP with its own layout. In this project, `ingest/make_ppd_tables.R` is the one file that reads the Department's files. Change it; every report re-renders.

Links

This project	`[REPO LINK]`
Posit Cloud space	`[SPACE LINK]`
PPD:2026 source files	ed.gov, Negotiated Rulemaking 2025-2026, AHEAD Session 2 materials
Quarto	quarto.org/docs/guide
R for Data Science, free	r4ds.hadley.nz
Auburn IR	ir@auburn.edu